

Vanni Ride

Pre-Development Master Specification

1. Project Overview

Project Name

Vanni Ride

Project Purpose

Vanni Ride is a platform designed to connect university students who need transportation or delivery services with students who own motorcycles and are willing to provide these services and earn income on a part-time basis.

The system acts as a connection platform between:

- **Customers** — students who need transportation or delivery services.
- **Riders** — students with motorcycles who provide transportation or delivery services.
- **Administrators** — authorized users who monitor and manage the platform.

Main Objective

The system should allow a customer to:

- Create a transportation or delivery request.
- Have the request made available to appropriate riders.
- Have a rider accept the request.
- Communicate with the assigned rider.
- Complete or cancel the request according to system rules.
- Record the cash payment status.
- Maintain a history of completed and cancelled requests.

2. System Scope

The group should confirm that these are the functions included in the first version.

Included

User Management

- Registration
- Login
- Logout
- Role-based access
- Profile management

Customer Functions

- Create ride/service request

- View request status
- View assigned rider
- Cancel request
- Chat with assigned rider
- View request history

Rider Functions

- View available requests
- Accept/reject requests
- Update availability
- Update request/ride status
- Chat with assigned customer
- View ride history

Ride Management

- Create request
- Make request available to riders
- Accept request
- Assign rider
- Update ride status
- Complete ride
- Cancel ride

Payment

- Cash only
- Record payment status

Communication

- Text-based chat
- Customer ↔ assigned rider only
- No image sending
- No file sending

Administration

- View/manage users
- View/manage rides
- Monitor cancellations
- Basic dashboard/statistics

3. Features Explicitly NOT Included

To prevent unnecessary development work, the group should confirm these are outside the scope.

- Online payment gateway

- Credit/debit card payment
- Bank integration
- Digital wallets
- Image sending in chat
- File sharing in chat
- Complex real-time tracking, unless already included in your proposal
- Complex fare estimation, unless specifically finalized in your proposal

This is important because the team should not independently add features simply because they seem useful.

4. User Roles

There are three main user roles.

4.1 Customer

A customer is a student who requires transportation or delivery.

Customer can:

- Register and log in
- Manage profile
- Create a ride/service request
- Enter pickup location
- Enter destination
- View request status
- View assigned rider
- Cancel a request according to allowed rules
- Chat with the assigned rider
- View request history
- View payment information

Customer cannot:

- Accept another customer's request
- Change ride status as a rider
- Access administration features
- Manage other users

4.2 Rider

A rider is a student who owns a motorcycle and provides services.

Rider can:

- Register and log in
- Manage profile
- Set availability
- View available requests
- View request details

- Accept a request
- Reject/ignore a request
- Update ride status
- Chat with the assigned customer
- View ride history
- View payment information related to their rides

Rider cannot:

- Access administration features
- Manage other users
- Accept the same ride twice
- Accept a ride already assigned to another rider

4.3 Administrator

The administrator manages and monitors the system.

Admin can:

- Log in
- View customers
- View riders
- View all ride records
- View completed rides
- View cancelled rides
- Monitor platform activity
- Manage user accounts if this is included in your finalized proposal

Admin cannot:

- Act as a rider while using admin functions
- Create ride requests through the admin panel

5. Main System Modules

The entire system can be divided into the following modules.

- Authentication & User Management
- Customer Module
- Rider Module
- Ride Management Module
- Payment Module
- Chat Module
- Admin Module
- Database / Testing / Integration

Every member should understand these modules, even if they are only responsible for one.

6. Complete System Architecture

The system should follow this layered structure:

Layer	Role / Responsibility
Users	Customer, Rider, and Admin — the three entry points into the system
Frontend	User interfaces used by each role, sending requests to the backend
Backend	Business logic and API layer, communicating with the database
Database	Stores all persistent system data

Users (Customer / Rider / Admin) → Frontend (UI) → Backend (API + Logic) → Database

Frontend — responsible for:

- Pages
- Forms
- Buttons
- Dashboards
- Displaying data
- Sending API requests
- Showing API responses/errors

Backend — responsible for:

- APIs
- Authentication
- Authorization
- Business rules
- Data validation
- Ride workflow rules
- Communication with database

Database — responsible for:

- Storing system data
- Relationships
- Constraints
- Data integrity

7. Team Responsibility Structure

Frontend Group — 2 Members

Member F1 — primary responsibility:

- Customer pages
- Rider pages
- Ride request UI

- Ride status UI
- Ride history
- Profiles

Member F2 — primary responsibility:

- Admin pages
- Chat UI
- Shared components
- Navigation
- Responsive design
- UI consistency

Both frontend members must agree on: folder structure, component naming, page layout, form validation style, and API calling method.

Backend Group — 2 Members

Member B1 — primary responsibility:

- Registration
- Login
- Authentication
- Authorization
- User roles
- User profiles
- Authentication middleware

Member B2 — primary responsibility:

- Ride request logic
- Rider acceptance
- Ride status management
- Cancellation
- Cash payment status
- Chat APIs

Both backend members must agree on: project structure, API naming, authentication approach, error response format, and validation approach.

Database / Testing / Integration Group — 2 Members

Member D1 — primary responsibility:

- Database design
- ER model
- Tables
- Relationships
- Constraints
- Queries

- Database updates
- Test/sample data

Member D2 — primary responsibility:

- Test plans
- API testing
- Frontend/backend integration
- Bug tracking
- Functional testing
- Integration testing

8. Core Ride Workflow

This must be agreed upon by everyone.

Standard Flow

1	Customer logs in
2	Customer creates a ride request
3	Status = REQUESTED
4	Available riders can view the request
5	A rider accepts the request
6	Rider is assigned — Status = ACCEPTED
7	Customer and rider can communicate
8	Rider arrives — Status = ARRIVED
9	Ride begins — Status = STARTED
10	Ride is completed — Status = COMPLETED
11	Cash payment is made — Payment = PAID

9. Ride Status Definitions

The team should use exactly the same status names everywhere. The database, backend, and frontend should all use the same values.

Status	Meaning
REQUESTED	Customer has created the request
ACCEPTED	A rider has accepted the request
ARRIVED	Rider has reached the pickup location
STARTED	Ride/service has started
COMPLETED	Service has finished
CANCELLED	Request was cancelled

10. Cancellation Workflow

This needs to be finalized before coding.

Customer cancels before acceptance

REQUESTED → CANCELLED

The rider is not assigned.

Customer cancels after acceptance

The team needs to confirm whether this is allowed. If allowed:

ACCEPTED → CANCELLED

The system should record:

- Who cancelled
- Cancellation time
- Cancellation reason, if required

After the ride starts

Recommended rule:

STARTED → Cancellation normally not allowed

Unless your proposal specifies another rule.

Important Database Requirement

Cancelled rides should **not** be deleted. They should remain in the system for:

- Ride history
- Admin monitoring
- Statistics
- Cancellation records

11. Payment Workflow

Vanni Ride uses **CASH ONLY**. The system does not process money electronically.

Suggested payment flow

1	Ride created
2	Payment status = PENDING
3	Ride completed
4	Customer pays rider in cash
5	Payment status = PAID

The group should decide: **who marks the payment as paid?**

Recommended options:

- Rider marks it as received.
- Both customer and rider confirm it.
- Admin can correct disputes.

For a student project, rider confirmation is the simplest approach, but the team should finalize this before implementation.

12. Chat Workflow

Chat should be restricted. **Rule:** a customer can only communicate with the rider assigned to their ride.

Customer → Chat for Ride #101 → Assigned Rider

Other users cannot access that conversation.

Chat features

- Send text messages
- Receive/display messages
- Timestamp
- Conversation history

Not included

- Images
- Files
- Voice messages
- Video calls

13. Ride Request Information

Before development, decide exactly what a customer enters. At minimum, a ride request includes:

- Customer
- Pickup Location
- Destination
- Request Type
- Request Time
- Current Status

Important decision to confirm

Because Vanni Ride supports both transportation and delivery, the group should confirm whether you will have two request types:

- TRANSPORT
- DELIVERY

If yes, the exact information required for each type must be decided. For example:

Transportation

- Pickup location
- Destination

Delivery (potentially)

- Pickup location
- Delivery destination
- Description of item

However, do not add item details if they were not included in your approved scope.

14. Rider Availability

The system should define how riders become available. Suggested statuses:

- AVAILABLE
- UNAVAILABLE
- BUSY

Rules

- Rider sets themselves to AVAILABLE.
- Only available riders should be shown available requests.
- When a rider accepts a ride, status becomes BUSY.
- After completion or cancellation, rider can become AVAILABLE again.

The group should confirm whether this automatic status change is part of your design.

15. Database Information Everyone Should Understand

The exact database design may change slightly, but everyone should understand the main relationships.

User relationships

- USER → Customer Profile
- USER → Rider Profile
- USER → Admin

Core ride relationship

Customer (creates) → Ride → Rider (assigned to)

Additional relationships

- Ride → Payment
- Ride → Messages
- Ride → Cancellation Record

16. Suggested Main Database Entities

USERS

Stores common information. Possible fields:

- user_id

- name
- email
- phone
- password_hash
- role
- account_status
- created_at

RIDERS

Stores rider-specific information. Possible fields:

- rider_id
- user_id
- availability_status

Additional fields should only be added if they are in your approved requirements.

RIDES

Stores ride/service information. Possible fields:

- ride_id
- customer_id
- rider_id
- request_type
- pickup_location
- destination
- status
- requested_at
- accepted_at
- started_at
- completed_at
- cancelled_at

PAYMENTS

- payment_id
- ride_id
- payment_method
- payment_status

MESSAGES

- message_id
- ride_id
- sender_id

- message_text
- sent_at

CANCELLATIONS

- cancellation_id
- ride_id
- cancelled_by
- reason
- cancelled_at

17. API Requirements

All groups should know the main API categories.

Category	Endpoints
Authentication	POST /api/auth/register POST /api/auth/login POST /api/auth/logout
Profile	GET /api/users/profile PUT /api/users/profile
Ride	POST /api/rides GET /api/rides GET /api/rides/:id PUT /api/rides/:id/accept PUT /api/rides/:id/status PUT /api/rides/:id/cancel
Chat	GET /api/rides/:id/messages POST /api/rides/:id/messages
Payment	GET /api/rides/:id/payment PUT /api/rides/:id/payment
Admin	GET /api/admin/users GET /api/admin/rides GET /api/admin/statistics

These are preliminary. The backend team should create the final API contract after the group agrees on the requirements.

18. API Data Standards

All teams should agree on naming conventions. For example, choose one style — do not mix pickupLocation, pickup_location, and pickup-location. The same rule should apply to database columns, API fields, and backend variables where practical.

Layer	Recommended convention	Example
Database	snake_case	pickup_location, ride_status, created_at
API / Frontend	camelCase or snake_case (pick one)	pickupLocation / pickup_location

Both are fine, but choose one and document it.

19. Authentication and Authorization Rules

The system should have secure role-based access:

Role	Access
CUSTOMER	Can access customer functions only
RIDER	Can access rider functions only
ADMIN	Can access admin functions

Important rule: the frontend should not be responsible for security. For example, hiding the Admin button does not stop someone from manually calling an admin API — the backend must verify the user's role.

20. Data Validation Rules

The group should decide basic validation.

Registration

- Required fields cannot be empty.
- Email must be valid.
- Password must meet the agreed minimum requirements.
- Email should be unique.

Ride Request

- Pickup location required.
- Destination required.
- Customer must be logged in.

Ride Acceptance

- Rider must be available.
- Ride must still be REQUESTED.
- Another rider must not already be assigned.

Messages

- Sender must belong to the ride.
- Message cannot be empty.

21. Error Handling Standards

The frontend should display understandable errors, for example:

- “Invalid email or password.”
- “This ride has already been accepted.”
- “You are not authorized to access this ride.”
- “Pickup location is required.”

Avoid only displaying technical errors to users. All backend APIs should return errors in a consistent format, for example:

```
{
  "success": false,
  "message": "This ride has already been accepted."
}
```

22. Folder and Code Organization

The exact structure depends on your chosen technology, but the group should agree on it. Example:

- vanni-ride/
 - frontend/ – components, pages, services, assets
 - backend/ – routes, controllers, models, middleware, config
 - database/ – schema, seed-data
 - documentation/

23. GitHub Workflow

Recommended branch structure:

- main
 - development
 - frontend-customer-rider
 - frontend-admin-chat
 - backend-auth
 - backend-ride
 - database
 - testing-integration

Basic rule:

Feature branch → Develop → Test → Pull Request → Review → development → main

Nobody should push unfinished code directly into main.

24. Integration Rules

The teams should not wait until the end — integration should happen feature by feature.

First integration

Registration → Login → Database

Second integration

Customer creates request → Backend receives request → Database stores request → Customer sees result

Third integration

Rider views request → Rider accepts → Database updates ride → Customer sees rider

Continue this process for every major feature.

25. Testing Responsibilities

Every developer must test their own work. However, the Database/Testing group coordinates overall testing.

Testing levels

- **Unit Testing** — test individual functions.
- **API Testing** — test backend endpoints.
- **Integration Testing** — test Frontend + Backend + Database together.
- **Functional Testing** — test complete user journeys.

User Flow Testing — example

1	Register Customer
2	Login
3	Create Request
4	Rider Accepts
5	Chat
6	Ride Starts
7	Ride Completes
8	Cash Payment

26. Definition of "Done"

A feature should not be considered finished just because the code was written. A feature is DONE only when all of the following are true:

- Code Completed
- Locally Tested
- Integrated
- Tested by another member
- No critical bugs

This is important for team coordination.

27. Common Project Rules

All six members should agree to these.

- **Rule 1** — Do not change database fields without informing the team.
- **Rule 2** — Do not change API formats without informing frontend/backend teams.
- **Rule 3** — Do not directly push unfinished code to the main branch.
- **Rule 4** — Do not add major new features without team agreement.
- **Rule 5** — Document important changes.
- **Rule 6** — Test before merging.
- **Rule 7** — Keep all team members aware of major changes.

28. Questions Your Team Must Confirm Before Coding

This is the most important part of this document. Your team should review and explicitly answer these questions.

Ride Requests

- Are there both transportation and delivery requests?
- What exact information is required for each?
- Can a customer create more than one active request?
- Who can see a new request?
- How is a rider selected?

Rider Availability

- How does a rider become available?
- Does accepting a ride automatically make the rider busy?
- When does the rider become available again?

Cancellation

- Can the customer cancel after acceptance?
- Can the rider cancel after acceptance?
- Is a cancellation reason required?
- Can a ride be cancelled after it has started?

Payment

- Who enters or confirms that cash was paid?
- Is a payment amount stored?
- Is the amount decided by the system or outside the system?

Chat

- When does chat become available?
- Does chat remain available after ride completion?
- Are messages retained permanently?

Administration

- Can an admin deactivate users?
- Can an admin modify ride records?
- What statistics are required?

Final Pre-Development Checklist

Before coding begins, every team member should agree that the following is finalized:

Project Scope

- Included features confirmed
- Excluded features confirmed

Users

- Customer permissions finalized
- Rider permissions finalized

- Admin permissions finalized

Ride System

- Ride workflow finalized
- Ride statuses finalized
- Cancellation rules finalized
- Rider availability rules finalized

Payment

- Cash-only rule confirmed
- Payment status workflow finalized
- Payment responsibility finalized

Communication

- Chat rules finalized
- Text-only rule confirmed

Technical

- Technology stack selected
- System architecture agreed
- Database structure approved
- API requirements identified
- Naming conventions agreed
- GitHub workflow agreed

Development

- Responsibilities assigned
- Integration schedule agreed
- Testing process agreed
- Definition of "Done" agreed